

Video 2.4: Real-World CVE Analysis and Exploitation

Duration: 8:30

Format: PPT (20%) / Terminal (35%) / Code Editor (25%) / Browser (20%)

Resolution: 1920x1080 @ 30fps

Prerequisites: Videos 2.1-2.3

SCRIPT BEGIN

[0:00-0:40] — Opening: Why CVEs Matter (PPT Slide)

SLIDE: Title: “Beyond Misconfiguration — Real Vulnerabilities”

Three CVEs highlighted:

- CVE-2019-5736 (runc) — Runtime binary overwrite
- CVE-2020-15257 (containerd) — Host networking escape
- CVE-2022-0185 (Linux kernel) — Filesystem context overflow

NARRATION:

“In the previous lessons, we exploited misconfigurations — privileged flags, socket mounts, capability grants. But what about properly configured containers? Can an attacker escape a default, non-privileged container? The answer is yes — through vulnerabilities in the container runtime, the kernel, or the orchestration layer. In this lesson, we’ll analyze three CVEs that allowed real container escapes from non-privileged configurations. For each one, we’ll understand the vulnerability, trace the exploit chain, and discuss how it was detected and patched.”

[0:40-2:45] — CVE-2019-5736: The runc Escape (Browser + Code Editor + Terminal)

BROWSER: Navigate to `https://nvd.nist.gov/vuln/detail/CVE-2019-5736`

NARRATION:

“CVE-2019-5736 — disclosed in February 2019, CVSS 8.6. This is the most significant container escape CVE ever published. It affected runc — the low-level container runtime that Docker, Kubernetes, and nearly every container platform depends on. Any container runtime using runc versions below 1.0-rc6 was vulnerable. That was effectively the entire container ecosystem.”

“Let me read the key detail from the advisory: ‘runc allows attackers to overwrite the host runc binary and consequently obtain host root access.’ The attacker crafts a malicious container — or gains shell access in a running container — and when `docker exec` is used to enter the container, the exploit overwrites the runc binary on the host.”

CODE EDITOR — CVE-2019-5736 Exploit Flow at 150% zoom:

CVE-2019-5736 – EXPLOIT CHAIN

=====

VULNERABILITY:

- runc did not properly handle /proc/self/exe during container exec
- /proc/self/exe is a symlink to the running binary
- Inside a container, /proc/self/exe points to the runc binary on the HOST

EXPLOIT FLOW:

1. Attacker creates a malicious container image with a custom entrypoint
 - OR gains shell access to a running container
2. Malicious entrypoint (or attacker) does the following:
 - a. Opens /proc/self/exe for writing
(This is a file descriptor to the HOST's runc binary)
 - b. Replaces the entrypoint with a script that:
 - Continuously tries to open /proc/self/exe
 - When docker exec runs, runc re-enters the container
 - The open file descriptor now points to host's runc binary
 - c. Writes a malicious payload to the file descriptor
(Overwrites host's /usr/bin/runc)
3. Next time ANY container operation runs on this host:
 - Docker calls the (now malicious) runc binary
 - Attacker's code runs AS ROOT ON THE HOST
 - Every subsequent container start/exec runs attacker code

IMPACT:

- Complete host compromise
- Persistent backdoor via binary replacement
- ALL containers on host are compromised
- Works from a default, non-privileged container

NARRATION:

“Let me trace this step by step. The vulnerability is in how runc handles the `/proc/self/exe` symlink. Inside a container, this symlink points back to the runc binary on the host — because runc is the process that created the container. The exploit opens this file descriptor, waits for a `docker exec` command, and then overwrites the host’s runc binary through that file descriptor. From that point forward, every container operation on the host executes the attacker’s code. This is a single container compromising the entire host — permanently — until the runc binary is replaced.”

TERMINAL:

```
# Check if your runc version is vulnerable
$ runc --version
runc version 1.1.12
commit: v1.1.12-0-g51d5e94
spec: 1.0.2-dev
```

“Current versions are patched. Versions below 1.0-rc6 were vulnerable. Let me show you the fix.”

```
# The fix: runc is now cloned to a memfd before exec
# Check the protection
$ grep -r "memfd_create\|proc_self_exe" /usr/bin/runc 2>/dev/null
# (binary file, but the fix uses memfd_create to create a sealed copy)
```

“The fix was elegant — runc now creates a sealed, in-memory copy of itself using `memfd_create` before entering the container. The container process can no longer access the original binary through `/proc/self/exe`. The symlink now points to the in-memory copy, which is sealed and cannot be written to.”

[2:45-4:45] — CVE-2020-15257: containerd Host Networking Escape (PPT + Code Editor + Terminal)

SLIDE: Title: “CVE-2020-15257 — containerd Escape via Host Networking”

Field	Detail
Affected	containerd < 1.3.9, < 1.4.3
CVSS	5.2 (Medium) — but practical impact is Critical
Vector	Containers with host network namespace
Requirement	Container running with <code>--net=host</code>
Impact	Full containerd API access → host compromise

NARRATION:

“CVE-2020-15257 targeted containerd — the container runtime that sits between Docker and runc. The vulnerability: when a container used the host network namespace with `--net=host`, it could access the containerd control socket — which is an abstract Unix socket that’s visible in the host network namespace. Through this socket, an attacker could call containerd’s API to start new containers with arbitrary configurations — including privileged containers with host filesystem mounts.”

CODE EDITOR — CVE-2020-15257 Exploit Flow:

CVE-2020-15257 – EXPLOIT CHAIN

=====

PREREQUISITE: Container running with --net=host

VULNERABILITY:

- containerd's gRPC API listens on an abstract Unix socket
- Abstract Unix sockets are NOT isolated by mount namespaces
- They ARE visible to any process in the same NETWORK namespace
- --net=host shares the host network namespace → socket is accessible

EXPLOIT FLOW:

1. Attacker is inside a container with --net=host
\$ docker run --rm -it --net=host alpine sh
2. Attacker locates the containerd abstract socket:
\$ cat /proc/net/unix | grep containerd
Found: @/run/containerd/containerd.sock
3. Attacker connects to containerd's gRPC API using ctr or custom code:
Abstract socket path: \0/run/containerd/containerd.sock
(leading null byte = abstract socket)
4. Through the API, attacker can:
 - a. List all containers and their configurations
 - b. Read container filesystem snapshots
 - c. Start a NEW privileged container with host mounts
 - d. Execute commands in any running container
5. Attacker starts privileged container:
\$ ctr run --privileged --mount type=bind,src=/,dst=/hostfs ...
6. Full host access achieved

KEY INSIGHT:

- This exploits a NAMESPACE CONFUSION – mount isolation doesn't apply to abstract Unix sockets; only network isolation does
- --net=host removes that last barrier

NARRATION:

“The key insight is namespace confusion. Abstract Unix sockets — unlike regular Unix sockets — are not isolated by mount namespaces. They live in the network namespace. When a container shares the host network namespace, it can access any abstract socket the host can — including containerd’s control API. It’s a subtle interaction between namespace types that created a security gap.”

TERMINAL:

```
# Demonstrate: default container cannot see host abstract sockets
$ docker run --rm alpine sh -c 'cat /proc/net/unix | wc -l'
# Very few entries
```

“In a default container with its own network namespace — very few Unix sockets visible.”

```
# With host networking – all host abstract sockets are visible
$ docker run --rm --net=host alpine sh -c 'cat /proc/net/unix | wc -l'
# Many more entries – all host sockets visible
```

“With `--net=host` — every abstract socket on the host is visible. This is the exposure window. The fix was to restrict containerd’s socket access using additional authentication, and to move away from abstract sockets.”

```
# Check your containerd version
$ containerd --version
containerd containerd.io 1.7.12
```

“Current versions are patched. But the lesson is important: `--net=host` is more dangerous than most people realize — it doesn’t just share the network stack, it exposes inter-process communication channels.”

[4:45-6:30] — CVE-2022-0185: Kernel Escape via Filesystem Context (PPT + Code Editor + Terminal)

SLIDE: Title: “CVE-2022-0185 — Linux Kernel Container Escape”

Field	Detail
Affected	Linux kernel 5.1 through 5.16.2
CVSS	8.4 (High)
Vector	Heap overflow in <code>legacy_parse_param()</code>
Requirement	<code>CAP_SYS_ADMIN</code> in current or initial user namespace
Impact	Kernel memory corruption → root on host

NARRATION:

“CVE-2022-0185 is a kernel-level vulnerability — the kind that can defeat even properly configured containers. It’s a heap-based buffer overflow in the `legacy_parse_param` function in the Linux kernel’s filesystem context handling. Any process with `CAP_SYS_ADMIN` — including unprivileged user namespace processes on some configurations — could trigger it.”

CODE EDITOR — CVE-2022-0185 Technical Analysis:

CVE-2022-0185 – TECHNICAL ANALYSIS

=====

VULNERABILITY:

- In Linux filesystem context API (fsconfig syscall)
- legacy_parse_param() function didn't validate string length
- Heap buffer overflow when parsing filesystem parameters
- Corrupts adjacent kernel heap objects

TRIGGER CONDITION:

- Call fsconfig() with a crafted parameter exceeding buffer bounds
- Requires CAP_SYS_ADMIN in the INITIAL user namespace
(or in a user namespace if unprivileged user namespaces are enabled)

EXPLOIT STRATEGY:

1. Spray the kernel heap with controlled objects
(e.g., msg_msg structures via msgsnd)
2. Trigger the overflow to corrupt adjacent heap objects
(overwrite msg_msg.m_ts field to enable OOB read)
3. Use corrupted msg_msg for information leak
(leak kernel addresses to defeat KASLR)
4. Perform a second heap spray + overflow
(target modprobe_path or cred structure)
5. Overwrite modprobe_path to point to attacker-controlled script
(OR directly modify current process credentials)
6. Trigger modprobe execution (e.g., by exec'ing unknown binary format)
→ Attacker's script runs as root on the host

TIMELINE:

- Jan 18, 2022: CVE published
- Jan 20, 2022: Patch released (kernel 5.16.2)
- Jan 25, 2022: Public exploit POC released
- 7 DAYS from disclosure to weaponized exploit

NARRATION:

“The exploit is sophisticated but well-documented. It uses heap spraying — filling kernel memory with controlled objects — then triggers the overflow to corrupt an adjacent message queue structure. This corrupted structure allows out-of-bounds reads, which leak kernel addresses to defeat KASLR. Then a second overflow targets `modprobe_path` — a kernel string that controls which binary runs when the kernel needs to load a module. By pointing it to an attacker-controlled script, the next time any process executes an unknown binary format, the attacker’s script runs as root. The entire chain — from initial overflow to root code execution on the host — is achievable from inside a container. The public exploit PoC was available within seven days of disclosure.”

TERMINAL:

```
# Check if your kernel is vulnerable
$ uname -r
5.15.0-92-generic
```

```
# Check the specific fix
$ grep -r "legacy_parse_param" /usr/src/linux-headers-$(uname -r)/ 2>/dev/null | head
-5
```

“The fix added proper length validation to `legacy_parse_param`. But the critical lesson is this: for kernel vulnerabilities, the only fix is kernel patching. No container configuration change, no Seccomp profile, no AppArmor policy can fix a bug in a kernel function that your allowed system calls depend on.”

```
# Mitigation: disable unprivileged user namespaces (reduces attack surface)
$ sysctl kernel.unprivileged_usersns_clone
# If this is 1, unprivileged users can create user namespaces
# Set to 0 to reduce CVE-2022-0185 exposure from non-privileged containers
$ sudo sysctl -w kernel.unprivileged_usersns_clone=0
```

“One mitigation — disable unprivileged user namespace creation. This reduces the attack surface because the exploit requires `CAP_SYS_ADMIN`, which unprivileged processes can obtain through user namespaces. Without user namespaces, only already-privileged containers are exposed.”

[6:30-7:30] — CVE Comparison and Lessons (PPT Slide)

SLIDE: Title: “Three CVEs — Three Attack Layers”

Dimension	CVE-2019-5736	CVE-2020-15257	CVE-2022-0185
Layer	Container runtime (runc)	Container manager (containerd)	Linux kernel
Requirement	Any container	--net=host	CAP_SYS_ADMIN
Complexity	Medium	Medium	High
Time to Exploit	Weeks	Weeks	7 days
Persistence	Binary replacement	New container	Kernel memory
Fix	Update runc	Update containerd	Patch kernel + reboot
Seccomp helps?	No (uses allowed syscalls)	No (network-level)	Partially (could block fsconfig)
Detection	Binary integrity monitoring	Socket access logging	Kernel exploit detection

NARRATION:

“Three CVEs, three different layers of the container stack. CVE-2019-5736 attacked the runtime — the binary that creates containers. CVE-2020-15257 attacked the container manager — the service that coordinates runtimes. CVE-2022-0185 attacked the kernel — the shared foundation everything runs on. Each required a different fix, a different detection strategy, and a different mitigation approach.

This is why container security requires defense-in-depth — you need protection at every layer because vulnerabilities can appear at any layer.”

[7:30-8:10] — Building a CVE Response Strategy (Code Editor)

CODE EDITOR — Container CVE Response Checklist:

Container CVE Response Playbook

Immediate Actions (< 1 hour)

1. Identify affected component (runtime, kernel, library)
2. Check installed versions across all hosts
3. Assess exposure: which containers/hosts are vulnerable?
4. Apply available mitigations (disable features, restrict access)

Short-term Actions (< 24 hours)

5. Test patches in staging environment
6. Deploy patches to production
7. If kernel CVE: schedule rolling reboots
8. Scan for indicators of compromise (IOCs)

Ongoing Actions

9. Subscribe to security advisories:
 - runc: github.com/opencontainers/runc/security
 - containerd: github.com/containerd/containerd/security
 - Linux kernel: lkml.org + distribution advisories
 - Docker: docs.docker.com/engine/security
10. Maintain a container bill of materials (BOM)
11. Automate vulnerability scanning in CI/CD pipeline
12. Run regular container escape detection tests

“Here’s a response playbook. Immediate — identify what’s affected and apply mitigations. Short-term — test and deploy patches, check for compromise. Ongoing — subscribe to advisories, maintain a bill of materials, automate scanning, and test your defenses. This playbook should be part of your incident response plan.”

[8:10-8:30] — Module 2 Wrap-Up (PPT Slide)

SLIDE: Title: “Module 2 — Complete”

What you now understand:

-  Privileged containers disable all security mechanisms
-  Host compromise from privileged containers in under 3 minutes
-  Multiple escape techniques from individual misconfigurations
-  Real CVEs that bypass even properly configured containers
-  CVE response and detection strategies

What’s next:

-  Module 3: System Call Filtering with Seccomp
-  You’ll build the defenses against these attacks

NARRATION:

“That completes Module 2. You’ve seen the offensive side — how containers are compromised through

misconfigurations and real vulnerabilities. You can demonstrate these techniques in lab environments, explain them to stakeholders, and detect them in production. Starting in Module 3, we shift to defense — building Seccomp profiles that restrict the system call surface we’ve been exploiting throughout this module. Complete the module assessment, and I’ll see you in Module 3.”

SCRIPT END

Post-Production Notes

- CVE browser pages should be pre-loaded and zoomed to 150%
- The exploit flow diagrams in the code editor should be revealed step by step
- For CVE-2019-5736, include an animated diagram showing the /proc/self/exe symlink chain
- For CVE-2022-0185, include a simplified heap memory diagram showing the overflow
- The CVE comparison table should animate column by column
- Add “⚠ LAB ENVIRONMENT” overlay during all terminal demonstrations
- End with module completion animation and assessment reminder
- The response playbook should be available as a downloadable resource
- Total module time check: Summary (2:30) + V1 (8:00) + V2 (8:00) + V3 (8:00) + V4 (8:30) = 35:00 — within 25-30 min target when accounting for summary being separate. Adjust if needed.
- NOTE: Combined lesson time is 32:30 (excluding summary). This slightly exceeds the 25-30 min target. Consider trimming Video 2.4 CVE analysis by 2-3 minutes if needed to stay within bounds, or accept the overage given the depth of material.